

PINN Cluster Scripts

What each script does, how the versioning works, and four end-to-end runthroughs.

Prerequisite: *Guide 1 – Using the Duke Compute Cluster* | Project root: `/hpc/group/fisherlab/ds555/pinn_code` |
Drafted 2026-08-02

SCOPE

Guide 1 covers Slurm itself — partitions, `sbatch`, arrays, dependencies. This guide assumes that and covers only what is specific to this repository: the two-tier script design, the `RUN_ROOT` methodology split, and the exact command sequences for the work currently in flight. Everything here is drawn from the scripts as they stand in `slurm/`; if a script changes, the script wins.

- | | |
|---|--|
| 1. One-time setup on the cluster | 8. Runthrough C — W0 audits (no training) |
| 2. The two-tier design: <code>run_</code> vs <code>submit_</code> | 9. Runthrough D — interactive debugging |
| 3. Script inventory | 10. Monitoring while jobs run |
| 4. <code>RUN_ROOT</code> — the v1/v2 split | 11. Recipes |
| 5. Reading <code>run_benchmark.sh</code> line by line | 12. Writing a new job script |
| 6. Runthrough A — W1 balancer, end to end | 13. Project-specific hazards |
| 7. Runthrough B — Stage 1 → Stage 2 | |

1. One-time setup on the cluster

```
# 1 — land on a login node
ssh ds555@dcc-login.oit.duke.edu

# 2 — get the code into GROUP space (never home — see Guide 1 §3)
cd /hpc/group/fisherlab/ds555
git clone <repo-url> pinn_code
cd pinn_code

# 3 — miniconda, also in group space
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh -b -p /hpc/group/fisherlab/ds555/miniconda3

# 4 — build the environment from the LINUX spec
source /hpc/group/fisherlab/ds555/miniconda3/etc/profile.d/conda.sh
conda env create -f environment_server.yml      # NOT environment.yml
conda activate pinn_env

# 5 — the log directory Slurm will not create for you
mkdir -p slurm/logs

# 6 — data in place?
ls -lh data/TSLA_2020_Split_Adjusted.csv
#   if absent, from your laptop:
#   scp data/TSLA_2020_Split_Adjusted.csv dcc:/hpc/group/fisherlab/ds555/pinn_code/data/

# 7 — prove the GPU stack works before committing GPU-hours
sbatch slurm/run_smoke_test.sh
squeue -u $USER
```

Step 4 deserves a note. `environment_server.yml` pins `torch==2.2.2+cu121`. If `nvidia-smi` on the GPU nodes reports a driver too old for CUDA 12.1, edit the `--index-url` to `cu118` and rebuild. The file's own header comment lists the mapping. Symptom of a mismatch: the environment builds fine, then `torch.cuda.is_available()` returns `False` inside a job that did request a GPU.

Step 5 is not optional and cannot be moved into a job script — Slurm opens the log file before your script's first line runs, so a missing `slurm/logs/` means the job dies leaving no log to explain itself. The `submit_*.sh` orchestrators call `mkdir -p slurm/logs` themselves, but a bare `sbatch` of a leaf script does not.

2. The two-tier design: run_ vs submit_

Every script in `slurm/` is one of two kinds, and the prefix tells you which.

Prefix	Invoked with	Runs where	Job
<code>run_*.sh</code>	<code>sbatch</code> <code>slurm/run_x.sh</code>	compute node	Leaf job. Has the <code>#SBATCH</code> header, activates conda, resolves its array index to a configuration, and calls one <code>python</code> entry point.
<code>submit_*.sh</code>	<code>bash</code> <code>slurm/submit_x.sh</code>	login node	Orchestrator. No <code>#SBATCH</code> header at all. Submits a smoke test, captures its ID, then submits the real array with <code>--dependency=afterok</code> , and prints the task→config map.

THE MOST COMMON MISTAKE

`sbatch slurm/submit_w1.sh` is wrong — that would queue the orchestrator as a job, and it would try to call `sbatch` from inside a compute node. Orchestrators are run with `bash`, directly on the login node. They finish in under a second because all they do is submit. The rule: **bash** the **submit_** ones, **sbatch** the **run_** ones.

The reason for the split is that the dependency chain has to be constructed somewhere, and constructing it requires knowing the smoke test's job ID — which only exists after `sbatch` returns. That is a login-node activity, and it is cheap. Meanwhile the leaf script stays a clean, self-contained unit you can submit by hand when you want to skip the gate.

3. Script inventory

Script	Tasks	Time	What it does
Orchestrators — bash these			
<code>submit_all.sh</code>	—	—	Stage 1: smoke → B0–B12 array (13 tasks).
<code>submit_stage2.sh</code>	—	—	Stage 2: smoke → 2×2 array (4 tasks). Requires Stage 1 complete.
<code>submit_w1.sh</code>	—	—	W1 balancer: smoke → 6-task array. The current active workstream.
Leaf jobs — sbatch these			
<code>run_smoke_test.sh</code>	1	0:30	Nov2020, 1500 epochs, one config. Verifies GPU + env + val-best snapshot pipeline.
<code>run_benchmark.sh</code>	13	4:00	Stage 1 workhorse. B0–B12: arch × mode × μ plus five loss-balancing ablations. 15000 epochs, 9 folds each.
<code>run_stage2_smoke.sh</code>	1	0:30	Validates both B10 and B12 warm-starts, C-Vol only, 1000 epochs.
<code>run_stage2.sh</code>	4	4:00	Stage 2: {B10, B12} warm-start × {cvol, avol}. 10000 epochs.
<code>run_w1_smoke.sh</code>	1	0:30	Exercises all three new balancer code paths in sequence, 1500 epochs each.
<code>run_w1_balancer.sh</code>	6	2:30	W1 falsification test. {no-warmup, warmup5000} × {relobralo, gradnorm_renorm, fixed}, on 3 folds.
<code>run_w0_audits.sh</code>	1	0:45	Analysis only, no training. Physics-fidelity audit + σ -vs-market-IV overlay over existing checkpoints.
<code>run_history_capture.sh</code>	2	1:00	Re-runs B6 and B10 on Nov2020 only, to capture per-epoch history dicts.

All of them request the same shape of node: `--partition=gpu-common --gres=gpu:2080:1 --cpus-per-task=4`, with 32 G for full runs and 16 G for smoke tests. That uniformity is deliberate — it means queue behaviour is predictable and you can compare timings across scripts.

4. RUN_ROOT — the v1/v2 split

This is the single most consequential convention in the repository, and the one most likely to cause silent damage if misunderstood. The full rationale is in `EXPERIMENTS.md`; the operational summary is here.

The project is mid-way through changing core methodology — the loss balancer, the selection protocol, and the arbitrage objective are all being replaced. Old results must stay comparable, so they are **frozen**, and new results go to a **separate tree**.

	v1 — FROZEN	v2 — ACTIVE
Loss balancing	Wang grad-norm, unbounded $\Sigma g/g_i$	ReLoBRaLo / augmented Lagrangian (W1)
Selection	per-fold val-best argmin	fixed-budget final-epoch (W3)
Arbitrage	measured only	trained against (W4)
Run outputs	<code>runs/walk_forward/</code> , <code>runs/stage2_*/</code>	<code>runs/v2/walk_forward/</code> , <code>runs/v2/stage2_*/</code>
Reports	<code>reports/v1/</code>	<code>reports/v2/</code>
Status	complete; matches <code>paper/paper.qmd</code>	in progress

The mechanism is one environment variable. Each leaf script reads `RUN_ROOT` with a default, and interpolates it into `--output_dir`:

```
RUN_ROOT="${RUN_ROOT:-runs}"           # run_benchmark.sh, run_stage2.sh → default v1
RUN_ROOT="${RUN_ROOT:-runs/v2}"       # run_w1_balancer.sh → W1 is new work, defaults v2

python run_walk_forward.py ... --output_dir ${RUN_ROOT}/walk_forward
```

So the older scripts reproduce v1 exactly when run bare, and are routed to v2 by prefixing the submission:

```
# v1 reproduction (what already exists – you rarely want this)
sbatch --array=0-12 slurm/run_benchmark.sh

# v2 – same grid, new methodology, separate tree
RUN_ROOT=runs/v2 sbatch --array=0-12 slurm/run_benchmark.sh
```

NEVER WRITE NEW METHODOLOGY INTO THE DEFAULT TREE

Forgetting the `RUN_ROOT=runs/v2` prefix does not error. It writes v2-methodology results into the v1 directory, where `build_master_table.py` will happily aggregate them alongside genuine v1 rows — producing a master table that silently mixes two methodologies. That corrupts every comparison downstream, including the manuscript's tables, and it is not obvious from the numbers. **Check the `Output:` line the script echoes before you walk away.**

Equally: never overwrite `reports/v1/`. It is the frozen snapshot the paper cites.

One subtlety in `run_stage2.sh`: it reads the warm-start checkpoint from `${RUN_ROOT}/walk_forward/...` — the *same* root it writes to. This is intentional. It means `RUN_ROOT=runs/v2 sbatch slurm/run_stage2.sh` warm-starts from v2 Stage 1 checkpoints, keeping the methodology consistent end to end. It also means v2 Stage 2 will fail if v2 Stage 1 has not been run — the checkpoint directory simply will not exist.

The non-PINN baselines (BS, GAM, laGP) are methodology-independent, so `results/{bs,gam,lagp}_baseline/` is shared and v2 reuses it. No need to re-run them.

5. Reading run_benchmark.sh line by line

Every leaf script in this repo follows the same five-part shape. Learn it once here and the rest are obvious.

Part 1 — the header

```
#SBATCH --job-name=pinn_benchmark
#SBATCH --partition=gpu-common
#SBATCH --gres=gpu:2080:1
#SBATCH --mem=32G
#SBATCH --cpus-per-task=4
#SBATCH --time=4:00:00
#SBATCH --output=slurm/logs/slurm_%A_%a.out
#SBATCH --error=slurm/logs/slurm_%A_%a.err
```

Note `%A_%a` rather than `%j`: array parent ID plus task index, so the 13 tasks write 13 separate logs. The 4-hour limit covers 9 folds $\times$ 15000 epochs on a 2080 with headroom.

Part 2 — environment

```
cd /hpc/group/fisherlab/ds555/pinn_code
source /hpc/group/fisherlab/ds555/miniconda3/etc/profile.d/conda.sh
conda activate pinn_env
```

Absolute `cd` first, because a batch job's working directory is not reliably where you submitted from. Then the explicit `source` — a non-interactive shell never reads `.bashrc`, so the `conda` function does not exist yet (Guide 1 §5.3).

Part 3 — output routing

```
RUN_ROOT="${RUN_ROOT:-runs}"
```

The `${VAR:-default}` form: use `RUN_ROOT` from the environment if set and non-empty, otherwise `runs`. This is what makes the v1/v2 switch a submission-time decision rather than a code edit.

Part 4 — the index lookup

```
ARCHS=(standard standard modified modified modified modified modified ...)
MODES=(physics hybrid physics physics physics hybrid hybrid ...)
MUS=( 1.0 1.0 0.5 0.75 1.0 0.5 0.75 ...)
FIXED_DATA=("" "" "" "" "" "" "" "" "" "1000.0" "" "" "")
WARMUP=(0 0 0 0 0 0 0 0 0 5000 5000 5000)

ARCH=${ARCHS[$SLURM_ARRAY_TASK_ID]}
MODE=${MODES[$SLURM_ARRAY_TASK_ID]}
MU=${MUS[$SLURM_ARRAY_TASK_ID]}
FD=${FIXED_DATA[$SLURM_ARRAY_TASK_ID]}
WU=${WARMUP[$SLURM_ARRAY_TASK_ID]}

EXTRA=""
[ -n "$FD" ] && EXTRA="$EXTRA --fixed_data_weight $FD"
[ "$WU" -gt 0 ] && EXTRA="$EXTRA --data_loss_warmup $WU"
```

Five parallel arrays, one column per task, positions aligned. The `EXTRA` accumulator handles optional flags: an empty `FIXED_DATA` entry contributes nothing, so only task 9 gets `--fixed_data_weight`, and only tasks 10–12 get `--data_loss_warmup`. This keeps the command line clean rather than passing sentinel values the Python side would have to interpret.

The B0–B12 grid it encodes:

Task	ID	Arch	Mode	μ	Extras / purpose
0	B0	standard	physics	1.00	naive baseline
1	B1	standard	hybrid	1.00	naive + market data
2	B2	modified	physics	0.50	μ sweep, physics only
3	B3	modified	physics	0.75	—
4	B4	modified	physics	1.00	—
5	B5	modified	hybrid	0.50	μ sweep, hybrid
6	B6	modified	hybrid	0.75	old Stage 2 default; the worst v1 drifter
7	B7	modified	hybrid	1.00	—
8	B8	modified	hybrid	0.00	fix #1 — kill RWF init blowup
9	B9	modified	hybrid	0.75	<code>--fixed_data_weight 1000</code> — decouple data from grad-norm
10	B10	modified	hybrid	0.75	<code>--data_loss_warmup 5000</code> — best v1 RMSE
11	B11	modified	hybrid	0.50	warmup 5000 — μ -down + warmup
12	B12	modified	hybrid	0.25	warmup 5000 — cleanest PDE surface

Part 5 — echo, then run

```
echo "Config: arch=$ARCH mode=$MODE  $\mu$ =$MU extras=[$EXTRA]"

python run_walk_forward.py \
  --arch $ARCH --mode $MODE --rwf_mu $MU \
  --epochs 15000 --val_months 1 --seed 42 \
  --output_dir ${RUN_ROOT}/walk_forward \
  $EXTRA
```

The `echo` of the *resolved* configuration is not decoration. It is the record that lets you confirm, from the log alone, that task 9 really did get `--fixed_data_weight 1000` — which is the only practical defence against a misaligned parallel array. Always echo the resolved config, never just the task index.

6. Runthrough A — W1 balancer, end to end

This is the currently active workstream. The question it answers: does *bounding* the loss balancer kill the λ_{data} runaway and stop hybrid mode degrading relative to physics? (See `docs/TRAINING_VALIDATION_DISCUSSION.md` P1 and `docs/TOD0.md` W1.)

Step 1 — submit

```
cd /hpc/group/fisherlab/ds555/pinn_code
git pull
bash slurm/submit_w1.sh
```

Output:

```
=== Submitting W1 smoke test (relobralo / gradnorm_renorm / fixed) ===
Smoke job ID: 2891044

=== Submitting W1 balancer array (6 tasks, depends on smoke) ===
W1 array job ID: 2891045

Monitor with:  squeue -u ds555
Watch logs:    tail -f slurm/logs/w1_2891045_0.out

Task 0: no-warmup    × relobralo
Task 1: no-warmup    × gradnorm_renorm
Task 2: no-warmup    × fixed
Task 3: warmup=5000 × relobralo
Task 4: warmup=5000 × gradnorm_renorm
Task 5: warmup=5000 × fixed
```

Both submissions happen instantly. The array sits `PD` with reason `Dependency` until the smoke test exits 0.

Step 2 — watch the gate

```
squeue -u $USER
tail -f slurm/logs/w1_smoke_2891044.out
```

The smoke test runs all three balancers in sequence on Nov2020 at 1500 epochs — chosen because 1500 exceeds `grad_norm_freq` (1000), so the balancer fires twice, and `--data_loss_warmup 200` makes the warmup→hybrid transition happen early enough to be exercised. It has `set -e`, so any balancer failing aborts the whole thing and the array never starts.

IF THE SMOKE FAILS

The array does not fail — it hangs in `PD` with `DependencyNeverSatisfied`, permanently. Read `slurm/logs/w1_smoke_*.err`, then:

```
scancel 2891045          # kill the orphaned array
# fix the problem, commit, then re-run: bash slurm/submit_w1.sh
```

Step 3 — the array runs

Six tasks, each: modified hybrid, $\mu=0.75$, 15000 epochs, folds Aug2020/Oct2020/Nov2020, `--track_test_curve` on. Those three folds are where B6's hybrid degradation was clearest; Oct additionally stresses the September validation mismatch. Outputs land in:

```
runs/v2/walk_forward/modified_hybrid_mu0.75_<balancer>/
runs/v2/walk_forward/modified_hybrid_mu0.75_warmup5000_<balancer>/
```

No control runs are needed — the existing v1 grad-norm runs of B6 (no warmup) and B10 (warmup) on the same folds *are* the control.

Step 4 — check the acceptance criterion

W1 has a specific pass condition, and it is not "RMSE improved". Per `docs/TOD0.md`:

1. **Weight trajectories PLATEAU.** The λ multipliers must converge rather than run away — this is the actual defect being fixed. Plot them from the saved history dicts.
2. **Hybrid stops degrading vs physics** on the same folds.

A run where RMSE improves but λ_{data} still climbs to 10^5 has not passed. The numerical-error defect is the target; a lucky RMSE is not evidence.

Step 5 — aggregate and compare

```
python scripts/build_master_table.py --runs_dir runs/v2 --output_dir reports/v2 --label v2
python scripts/compare_methodologies.py
```

Both are light enough for the login node. Then pull the results down:

```
# from your laptop
rsync -avz dcc:/hpc/group/fisherlab/ds555/pinn_code/reports/ ./reports/
```

7. Runthrough B — Stage 1 → Stage 2

The full pipeline, as originally executed for v1. To reproduce it under the new methodology, prefix every `sbatch` with `RUN_ROOT=runs/v2`.

```
bash slurm/submit_all.sh
├─ sbatch run_smoke_test.sh          → job S      (~5 min)
└─ sbatch --array=0-12
    --dependency=afterok:S
    run_benchmark.sh                → array A    (13 × up to 4 h)
    │
    ▼ (wait for A to finish – check with sacct)
bash slurm/submit_stage2.sh
├─ sbatch run_stage2_smoke.sh        → job S2    (~10 min)
└─ sbatch --array=0-3
    --dependency=afterok:S2
    run_stage2.sh                   → array B    (4 × up to 4 h)
    │
    ▼
python scripts/build_master_table.py
```

Stage 1

```
bash slurm/submit_all.sh

# confirm all 13 tasks completed before going further
sacct -j <array-id> --format=JobID,JobName%20,State,Elapsed | grep -v "\.batch\|\.extern"
```

Stage 2 warm-starts from Stage 1 checkpoints, so a Stage 1 task that failed means the corresponding Stage 2 task will fail too — with a missing-directory error rather than anything informative. Verify completion first; re-run any failed indices (§11.1).

Stage 2

```
bash slurm/submit_stage2.sh
```

Four tasks — two warm-start sources crossed with two volatility parameterizations:

Task	Warm-start	Vol type	Output
0	B10 ($\mu=0.75$) — best RMSE	cvol	runs/stage2_B10/cvol
1	B10 ($\mu=0.75$)	avol	runs/stage2_B10/avol
2	B12 ($\mu=0.25$) — cleanest PDE	cvol	runs/stage2_B12/cvol
3	B12 ($\mu=0.25$)	avol	runs/stage2_B12/avol

`cvol` is the multiplicative parameterization, `avol` the direct one. Both use `--pricing_lr 1e-4 --vol_lr 1e-3`: the pricing net is fine-tuned gently since it arrives pre-trained, while the vol net starts fresh and needs a faster rate.

8. Runthrough C — W0 audits (no training)

`run_w0_audits.sh` is the one script that trains nothing. It loads existing v1 checkpoints and evaluates them on a dense grid: a physics-fidelity audit (PINN vs analytic Black–Scholes, plus arbitrage severity) and a σ_θ -versus-market-IV overlay asking whether Stage 2 learned real volatility.

```
# defaults: the data-rich folds
sbatch slurm/run_w0_audits.sh

# override the subsets via environment variables
FIDELITY_FOLDS=all sbatch slurm/run_w0_audits.sh
OVERLAY_FOLDS=Nov2020 sbatch slurm/run_w0_audits.sh
FIDELITY_CONFIGS="modified_physics_mu0.75" FIDELITY_FOLDS=Nov2020 sbatch slurm/run_w0_audits.sh
```

Same `${VAR:-default}` idiom as `RUN_ROOT`, applied to fold and config subsets. Outputs go to `reports/diagnostic/` — CSVs merge across re-runs, PNGs overwrite.

These take minutes rather than hours, which makes them tempting to run on the login node. Don't: they load checkpoints and evaluate on a dense grid, which is exactly the kind of memory-hungry work login nodes are not for. The script's own header documents the interactive alternative if you want to iterate on them.

9. Runthrough D — interactive debugging

When you are changing training code and want a fast edit-run-crash loop:

```

# On the login node, start tmux FIRST so a dropped connection doesn't kill the allocation
tmux new -s pinn

srun --partition=gpu-common --gres=gpu:2080:1 \
    --mem=16G --cpus-per-task=4 --time=1:00:00 --pty bash

# now on a compute node
cd /hpc/group/fisherlab/ds555/pinn_code
source /hpc/group/fisherlab/ds555/miniconda3/etc/profile.d/conda.sh
conda activate pinn_env
python -c "import torch; print('CUDA:', torch.cuda.is_available())"

# tiny run - one fold, few epochs, transient output dir
python run_walk_forward.py \
    --arch modified --mode hybrid --rwf_mu 0.75 \
    --balancer relobralo --folds Nov2020 --epochs 300 \
    --data_loss_warmup 50 --output_dir runs/v2/smoke --track_test_curve

exit          # release the GPU. Detach tmux with Ctrl-b d if you want to keep the shell.

```

Use `runs/v2/smoke` for this — it is gitignored, so scratch runs never pollute the results tree or show up in a commit. The same applies to `runs/smoke/`, `runs/stage2_smoke/`, and `runs/v2/stage2_smoke/`.

10. Monitoring while jobs run

```

# the queue, refreshing
watch -n 30 "squeue -u $USER"

# follow one array task's stdout
tail -f slurm/logs/w1_2891045_0.out

# which tasks of an array are done, and how they ended
sacct -j 2891045 --format=JobID,State,Elapsed,MaxRSS

# resource efficiency, once finished - feeds your next --mem/--time choice
seff 2891045_0

# quick scan of all six tasks for errors
grep -l "Error\|Traceback" slurm/logs/w1_2891045_*.err

# confirm output actually landed where intended
ls -la runs/v2/walk_forward/

```

The log naming conventions, so you know what to `tail`:

Pattern	Produced by
slurm/logs/slurm_%A_%a.out	run_benchmark.sh — Stage 1 array
slurm/logs/s2_%A_%a.out	run_stage2.sh
slurm/logs/w1_%A_%a.out	run_w1_balancer.sh
slurm/logs/w0_%j.out	run_w0_audits.sh (single job, so %j)
slurm/logs/smoke_%j.out , w1_smoke_%j.out , s2_smoke_%j.out	the smoke tests
slurm/logs/hist_%A_%a.out	run_history_capture.sh

11. Recipes

11.1 Re-run only the failed tasks of an array

```
sacct -j 2891045 --format=JobID,State | grep -E "FAILED|TIMEOUT"
# suppose tasks 2 and 5 failed:
RUN_ROOT=runs/v2 sbatch --array=2,5 slurm/run_w1_balancer.sh
```

Index N always maps to the same configuration, so this reproduces exactly the failed cells. Remember to carry the `RUN_ROOT` prefix — a re-run without it writes to the wrong tree.

11.2 One config, no array

```
sbatch --array=10 slurm/run_benchmark.sh      # just B10, via the array machinery
```

A single-element array keeps the index-lookup path intact, so you get exactly the documented configuration rather than a hand-typed approximation of it.

11.3 Override a resource limit without editing the script

```
sbatch --time=8:00:00 --mem=64G --array=0-5 slurm/run_w1_balancer.sh
```

Command-line flags beat `#SBATCH` lines. This is how you rescue a `TIMEOUT` without a commit-and-revert cycle on a version-controlled script.

11.4 Add a new configuration to the grid

Append one element to **every** parallel array, then extend the range:

```
ARCHS=(... modified)      # now 14 entries
MODES=(... hybrid)        # now 14 entries
MUS=(... 0.10)             # now 14 entries
FIXED_DATA=(... "")       # now 14 entries – the empty string matters
WARMUP=(... 5000)          # now 14 entries
```

```
sbatch --array=13 slurm/run_benchmark.sh      # run only the new cell
```

Then update the header comment's task→config map. The failure mode of skipping an array is silent: a missing element expands to empty, and the run proceeds with a subtly wrong command line. Verify from the log's `Config:` echo before trusting the result.

11.5 Run a v2 sweep of the full Stage 1 grid

```
RUN_ROOT=runs/v2 sbatch --array=0-12 slurm/run_benchmark.sh
RUN_ROOT=runs/v2 sbatch --array=0-3 slurm/run_stage2.sh      # after the above completes
python scripts/build_master_table.py --runs_dir runs/v2 --output_dir reports/v2 --label v2
python scripts/compare_methodologies.py
```

11.6 Throttle a wide array

```
sbatch --array=0-12%4 slurm/run_benchmark.sh    # at most 4 running at once
```

Worth doing on `gpu-common` during busy periods — 13 simultaneous GPU requests is a lot to ask of a shared pool, and throttling costs you little since they queue anyway.

11.7 Email on completion

```
sbatch --mail-type=END,FAIL --mail-user=ds555@duke.edu --array=0-5 slurm/run_w1_balancer.sh
```

With an array, `--mail-type=END` emails per task — thirteen emails from a Stage 1 sweep. `FAIL` alone is usually the humane setting.

12. Writing a new job script

Copy an existing leaf script rather than starting blank; the boilerplate is the part that is easy to get subtly wrong. A minimal template that matches this project's conventions:

```
#!/bin/bash
# =====
# <What question does this answer? Link the doc section.>
#
# Task → config:
# 0: ...
# 1: ...
#
# Submit: RUN_ROOT=runs/v2 sbatch --array=0-1 slurm/run_wX.sh
# ACCEPTANCE: <what result would falsify the hypothesis?>
# =====

#SBATCH --job-name=pinn_wX
#SBATCH --partition=gpu-common
#SBATCH --gres=gpu:2080:1
#SBATCH --mem=32G
#SBATCH --cpus-per-task=4
#SBATCH --time=2:30:00
#SBATCH --output=slurm/logs/wX_%A_%a.out
#SBATCH --error=slurm/logs/wX_%A_%a.err

cd /hpc/group/fisherlab/ds555/pinn_code
source /hpc/group/fisherlab/ds555/miniconda3/etc/profile.d/conda.sh
conda activate pinn_env

RUN_ROOT="${RUN_ROOT:-runs/v2}"          # new work defaults to v2

PARAM_A=(val0 val1)
A=${PARAM_A[$SLURM_ARRAY_TASK_ID]}

echo "Job $SLURM_JOB_ID task $SLURM_ARRAY_TASK_ID on $SLURMD_NODENAME"
echo "GPU: $(nvidia-smi --query-gpu=name --format=csv,noheader 2>/dev/null)"
echo "Config: param_a=$A"
echo "Output: ${RUN_ROOT}/walk_forward"

python run_walk_forward.py --some_flag $A --output_dir ${RUN_ROOT}/walk_forward

echo "Done: $(date)"
```

The conventions worth preserving, and why:

- **A header comment stating the question and the acceptance criterion.** The scripts in `slurm/` do this consistently, and it is what makes them readable a year later. `run_w1_balancer.sh` is the model — it names the hypothesis, the control, the folds and why those folds, and what result would count as a pass.
- **New work defaults** `RUN_ROOT` to `runs/v2`. Then forgetting the prefix is harmless rather than corrupting.
- **Echo the resolved config, not the index.** Your only defence against misaligned arrays.
- **Put new balancers and penalties behind CLI flags** (per `CLAUDE.md`), so completed runs stay reproducible when defaults change.
- **Write a smoke test alongside it** and gate the array with `afterok`. A 6-task array at 2.5 hours is 15 GPU-hours; a five-minute gate is cheap insurance.

13. Project-specific hazards

Hazard	Why it bites, and what to do
<code>sbatch</code> -ing a <code>submit_*.sh</code>	Orchestrators have no <code>#SBATCH</code> header and call <code>sbatch</code> themselves. Use <code>bash</code> .
Missing <code>RUN_ROOT=runs/v2</code>	Silently mixes methodologies in the v1 tree and corrupts the master table. Check the <code>Output:</code> line in the log.
Overwriting <code>reports/v1/</code>	Frozen snapshot the manuscript cites. Direct new aggregations to <code>reports/v2/</code> with <code>--output_dir</code> .
Stage 2 before Stage 1	Warm-start checkpoint directory won't exist. Verify all 13 Stage 1 tasks are <code>COMPLETED</code> first.
v2 Stage 2 off v1 Stage 1	<code>run_stage2.sh</code> reads warm-starts from the same <code>RUN_ROOT</code> it writes to — by design. Run v2 Stage 1 first, or you get a missing-directory error.
Misaligned parallel arrays	Missing element → empty string → wrong or malformed command line, no error. Count every array when extending; read the <code>Config:</code> echo.
Using <code>environment.yml</code>	macOS-only; won't resolve on Linux. Always <code>environment_server.yml</code> .
CUDA version mismatch	Env builds, then <code>torch.cuda.is_available()</code> is <code>False</code> . Match the wheel's <code>--index-url</code> to the node driver.
Treating <code>track_test_curve</code> as selection	Test data is sacred (<code>CLAUDE.md</code>): each test month is evaluated once per fold. <code>--track_test_curve</code> is diagnostic only — it informs the W3 protocol decision, never model selection.
Missing <code>slurm/logs/</code>	Job dies with no log at all. <code>mkdir -p slurm/logs</code> . The orchestrators do it; a bare <code>sbatch</code> does not.
Scratch runs in the results tree	Use <code>runs/v2/smoke</code> — gitignored. <code>runs/walk_forward/</code> is committed.

BEFORE YOU SUBMIT ANYTHING EXPENSIVE

- `git pull` — the cluster copy is current
- `mkdir -p slurm/logs`
- Decide `RUN_ROOT` : is this v1 reproduction or v2 methodology?
- Smoke test first, gated with `--dependency=afterok`
- Confirm the log's `Config:` and `Output:` lines match your intent
- `seff` the first completed task and adjust limits for the rest

Related documents. `EXPERIMENTS.md` — the v1/v2 output policy in full. `docs/TOD0.md` — live status and the W0–W7 workstreams. `docs/HYBRID_PARETO_PLAN.md` — plan of record; §0 locked decisions, §4 sequencing. `docs/TRAINING_VALIDATION_DISCUSSION.md` — the P1–P5 defect analysis that motivates W1. `CLAUDE.md` — code conventions and diagnostic requirements.